RK2118 Secure Fault问题排查

文件标识：RK-KF-YF-C16

发布版本：V1.0.0

日期：2024-08-07

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文介绍了RK2118的Secure Fault调试方法。

产品版本

芯片名称	内核版本
RK2118	RT-Thread V4.1.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	陈谋春	2024-08-07	初始版本

目录

RK2118 Secure Fault问题排查

1. 概述
2. 调试原理
3. 实际用例
   - 3.1 触发Secure Fault
   - 3.2 根据CONTROL_NS查找SP位置
   - 3.3 回溯调用栈

1. 概述

RK2118的Star-MC1支持两种状态切换：安全和非安全，如果在非安全状态下去访问安全地址，就会触发Secure Fault，当前TF-M的处理是打印一些调试信息，然后直接复位。另外需要注意的是，由于当前配置的安全地址空间是包含0地址的，所以软件的空指针访问也会触发Secure Fault。

2. 调试原理

在Secure Fault的处理函数中，会包含非状态状态的各个重要寄存器，如：CONTROL_NS、MSP_NS和PSP_NS，根据这些寄存器，我们可以找到非安全的栈位置，同时结合符号表就可以回溯调用栈，定位出大致的问题代码。

3. 实际用例

下面就以一个io命令故意访问0地址为例，演示如何回溯调用栈，具体步骤如下：

3.1 触发Secure Fault

在设备的控制台输入如下命令：

```
# 用io命令访问0地址
io -4 0
# 会得到如下的调试输出
00000000: PSPLIMT:0x00200EE0
MSPLIMT:0x00200400
SAU SFSR:0x00000048
        [6] SFARVALID Secure fault address valid:0x00000000
        [3] AUVIOL Attribution unit violation flag.
SP_NS:0x3022FFE0
MSP_NS:0x3022FFE0
MSPLIM_NS:0x3022FC00
PSPLIM_NS:0x00000000
PSP_NS:0x30222BE0
PRIMASK_NS:0x00000000
BASEPRI_NS:0x00000000
FAULTMASK_NS:0x00000000
CONTROL_NS:0x00000002
dump PSP_NS:0x30222BE0
        0x1107F81C
        0x00000001
        0x00000000
        0x50220000
        0x11079245
```

```
0x1106BCB3
0x1106BE1A
0x61000000
0x00000000
0xFFFFFFBC
0x00000000
0x00000000
0x302219FC
0x30221BB9
0x30216960
0x30221BB9
0x00000007
0x1106BAC9
0x30221BB2
0x30222C40
0x00000002
0x11085980
0xDEADBEEF
0x11076A15
0x30221BB2
0x30221BB5
0x30221BB8
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x302219FC
0x30218FA0
0x00000002
0x00000000
0x30218F18
0x110828EC
0x30221BB2
0x11076F93
0x00000000
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0xDEADBEEF
0x11072035
0x23232323
0x1EA01EA1
0x30218CB4
0x302230D8
0x30221C9C
0x00000000
0x00000000
0x30222CD8
0x000E000E
0x00000000
0x0D0DE19C
```

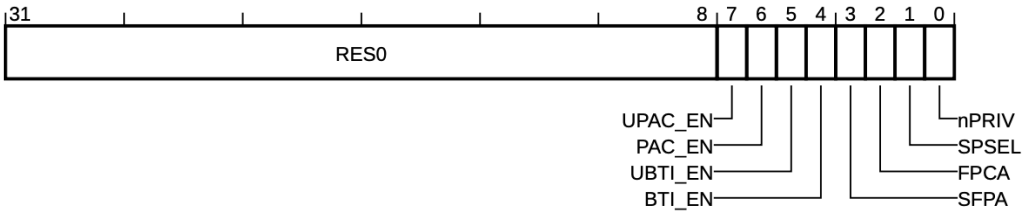
```
0x6F690D0D
0x20342D20
0x00000D30
0x00000000
0x00000000
0x00000000
```

```
        0x00000000
        0x00000000
        0x00000000
        0x00000000
        0x00000000
        0x00000000
        0x00000000
dump MSP_NS:0x3022FFE0
        0x30230000
        0x10000000
        0xE0000000
        0x00000003
        0x00000002
        0x110749BB
        0x30214A52
        0x21000000
        0xA65D5E16
        0x443AEB6C
        0xE106C4EE
        0x1321D707
        0x545A860B
        0x0B1E0292
        0xE2035183
        0x084812E4
        0xF7EFFD73
        0xE77BEFFF
        0xE7947FFF
        0xB41E75FF
        0x3FC3AE3E
        0x5BF5E267
        0xFFEDBD5D
        0xAED2B07A
        0x06303001
        0x125A2036
        0x9EB7D0B7
        0x9154C1A8
        0xB0691538
        0xCE16A6BA
        0x7F1E1092
        0x94044E04
        0x57BDA86B
        0x73B71AD8
        0xF7C03A93
        0xED3FDF77
        0x4D6FEAFA
        0x67AEEAFE
        0x5FEF7EF9
        0xF576F8C6
        0x206478D4
        0x27284341
        0x28043498
        0x165716A2
        0x480CEEF0
        0x7AB88C04
        0x15914011
        0x10095BF0
        0xDCDC5E34
        0x1F73D5F7
```

```
        0x9EE4E79F
        0x7DF7F739
        0x7C62EF7B
        0xFC1C7833
        0xFF9F7F9B
        0xEFDF1F87
        0x28780A18
        0x580581C4
        0x004EE99C
        0x2454D886
        0x04A30515
        0xE554C410
        0x052CE11E
        0xE044EA0D
        0x7970DBE4
        0x66F8DAC1
        0xF78DFFE2
        0xB822ADFB
        0xA9BED50F
        0x6F31E2FE
        0x672FD3BB
        0x2F65D38E
```

3.2 根据CONTROL_NS查找SP位置

CONTROL_NS寄存器的定义如下：



我们只需要关心SPSEL就好，这个bit定义如下：

- 0：Use SP_main as the current stack，对应了调试输出中的MSP_NS
- 1：In Thread mode use SP_process as the current stack，对应了调试输出中的PSP_NS

结合上面的调试输出我们可以知道：触发异常的代码用的是SP_process ，也就是当时的SP位置是0x30222BE0。

3.3 回溯调用栈

回溯调用栈之前先要知道代码段的地址范围，这个可以通过如下命令得到：

```
readelf -S rtthread.elf | grep ".text"
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 2] .sram.text        PROGBITS        30210834 001834 00438c 00  AX  0   0  4
  [ 3] .text             PROGBITS        110653c0 0063c0 020620 00  AX  0   0  8
  [10] .ddr.text         PROGBITS        a0100000 027f70 000000 00   W  0   0  1
```

从上面的信息可知，总共有三个代码段：

- .sram.text：放在sram中的代码段，地址范围：0x30210834 ~ 0x30214bc0
- .text：主代码段，地址范围：0x110653c0 ~ 0x110859e0
- .ddr.text：放在ddr中的代码段，size等于0，也就是当前没用到

上面的例子是开XIP的固件，有三个代码段，如果是XIP OFF的固件，就只有一个主代码段，会更简单一些。

得到代码段地址范围后，剩下的就是重复工作了：遍历栈中的每一个word，检查其是否属于合法指令地址，筛选条件如下：

1. 由于是Thumb指令集，所以合法地址必是一个奇数
2. 合法地址必然处于某一个代码段范围内

根据上面的条件，我们可以找到的合法指令地址如下：0x11079245 0x1106BCB3 0x1106BAC9，反编译调用栈命令如下：

```
arm-none-eabi-addr2line -e rtthread.elf 0x11079245 0x1106BCB3 0x1106BAC9
/home/rk/sdk/new_rtt/components/drivers/serial/serial.c:872
/home/rk/sdk/new_rtt/bsp/rockchip/common/drivers/io.c:56
/home/rk/sdk/new_rtt/bsp/rockchip/common/drivers/io.c:125
```

找到代码发现最后是在rt_serial_write函数，即正在做打印，这和实际预期不是完全重合，这个主要是因为cache的影响，0地址目前也是落在cacheable的地址空间，所以0地址访问不会马上触发Fault，最后回溯的调用栈也就不会很精准，只能是大致范围了。